

Wrap-up Week 5 + HTML Stuff

Theming, Fonts, & Dashboards

Maithreyi Gopalan

Week 6

Agenda

- Fonts w/ggplot2
- Flexdashboards (R Markdown)
- Quarto Dashboards (the modern approach)
 - What's new in 2025: `_brand.yml`, value boxes, light/dark mode
 - You'll build one step-by-step in Lab 6
- Some customization w/CSS (including fonts)
 - If we don't get to this today, we'll come back to it on Week 9
- Lab 4 and 5 review
- Quick refresher on troubleshooting html rendering

Fonts

General advice

- Match your plot fonts to your text body font
- Use different fonts to distinguish things
 - Specifically code
 - Consider for different heading levels
- **Always** choose a sans-serif font for code
- Explore and try - it makes a big impact on the overall look/feel
- Try not to get sucked into too deep of a rabbit hole

{ragg}

```
#install.packages("ragg")
```

Alternative device to Cairo, png, etc.

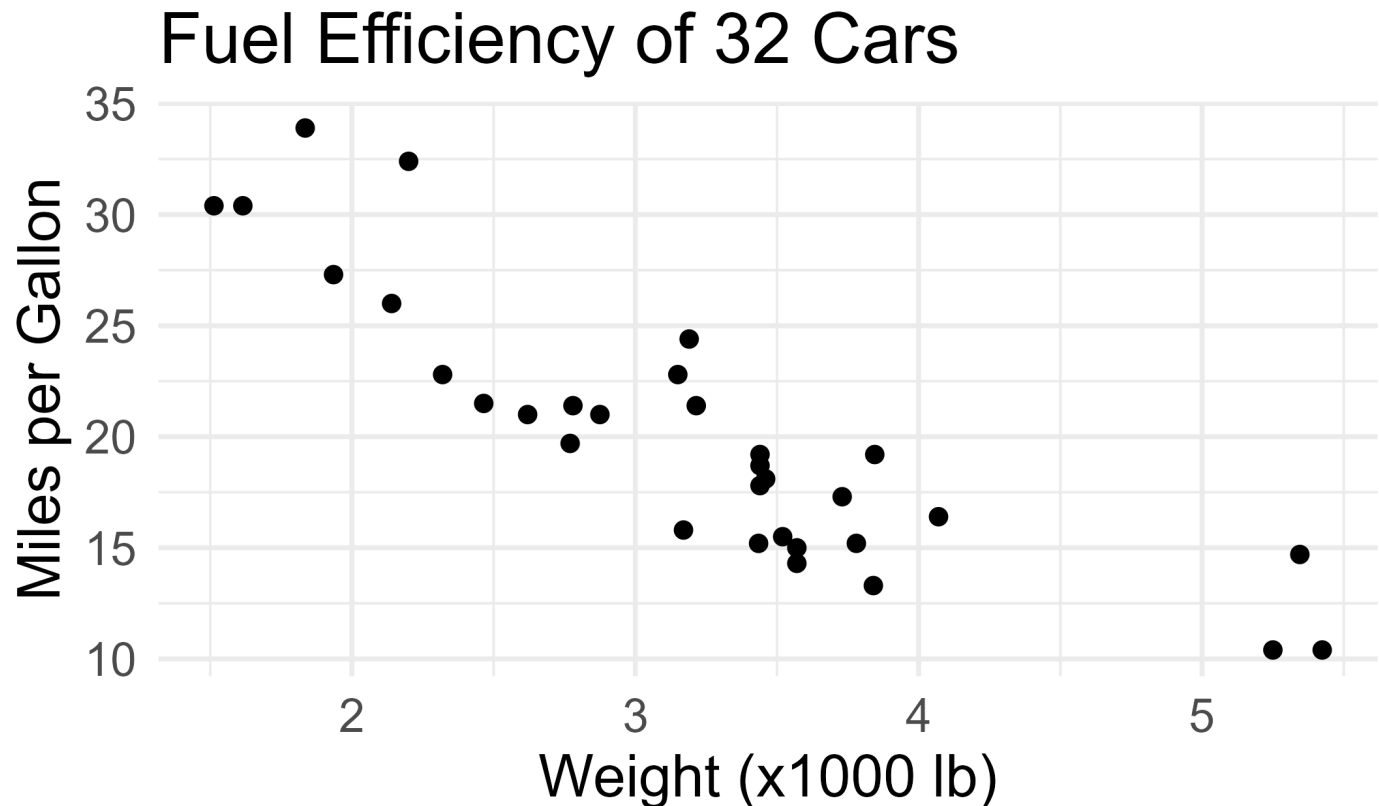
See the announcement [here](#)

After install, be sure to set *Global Options > General > Graphics* to *AGG*

Use with RMarkdown with `knitr::opts_chunk$set(dev = "ragg_png")`

Will automatically detect fonts you have installed on your computer

```
ggplot(mtcars, aes(wt, mpg)) +  
  geom_point() +  
  labs(title = "Fuel Efficiency of 32 Cars",  
        x = "Weight (x1000 lb)",  
        y = "Miles per Gallon") +  
  theme(text = element_text(family = "Luminari", size = 30))
```

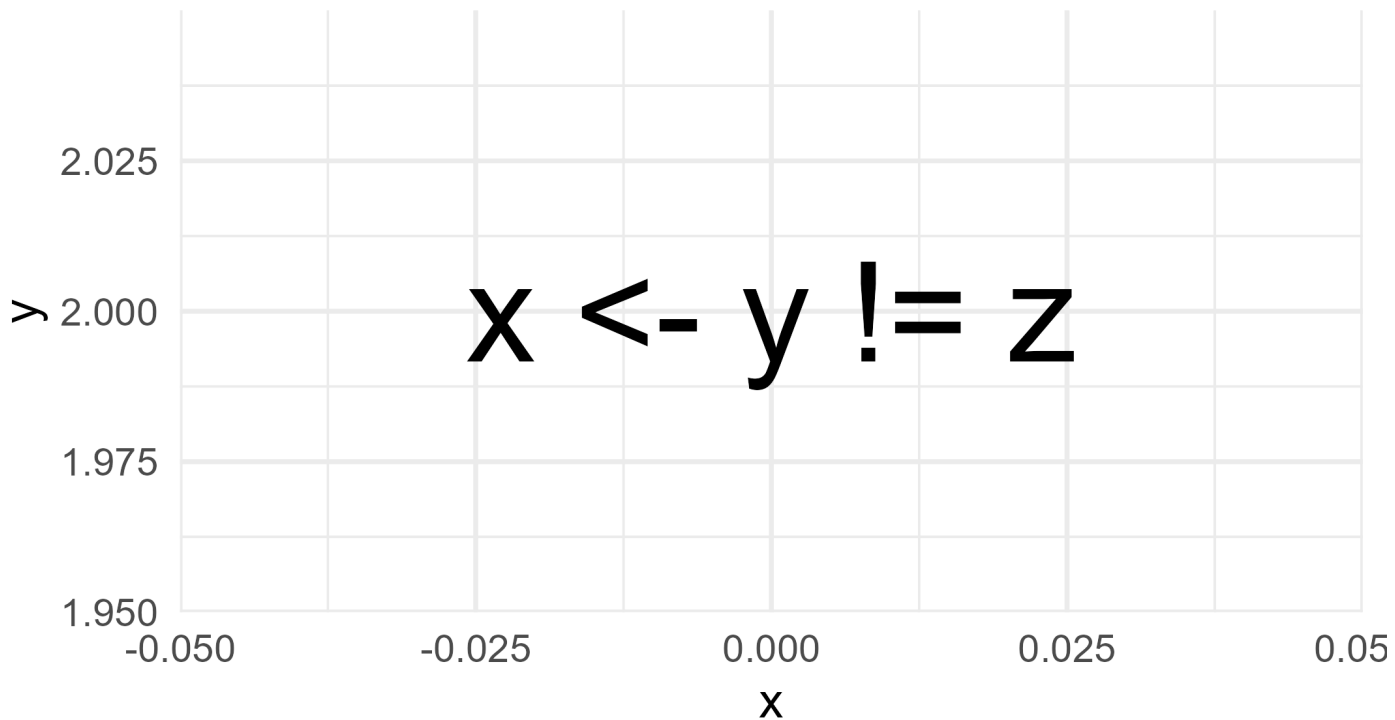


Support for lots of things!

Ligatures and font-awesome icons

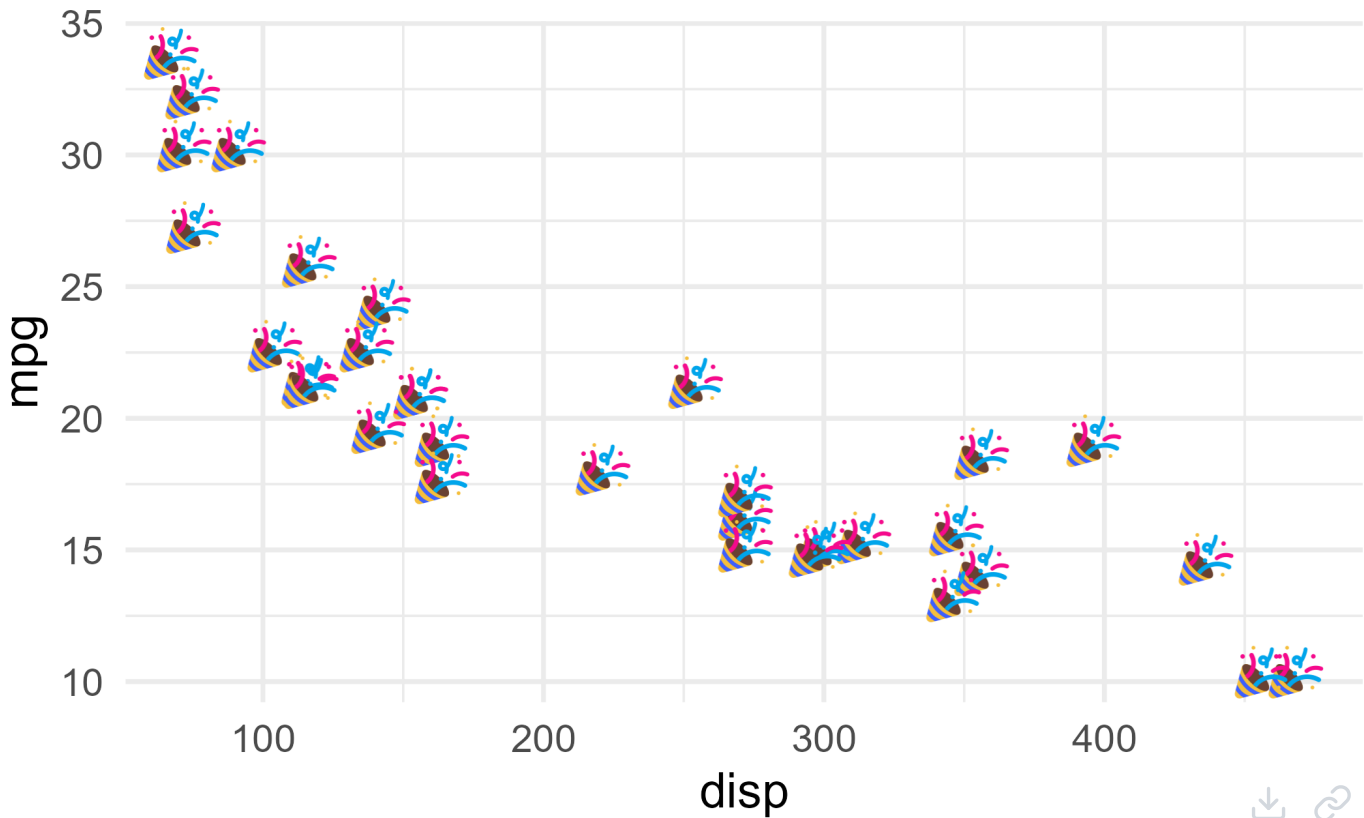
```
ggplot() +  
  geom_text(  
    aes(x = 0, y = 2, label = "x <- y != z"),  
    family = "Fira Code"  
  ) +  
  labs(title = "twitter") +  
  theme(  
    plot.title = element_text(  
      family = "Font Awesome 5 brands"  
    )  
  )  
)
```

twitter



emojis

```
ggplot(mtcars, aes(displacement, mpg)) +  
  geom_text(label = "🎉",  
            family = "Apple Color Emoji",  
            size = 10)
```



Google fonts

<https://fonts.google.com>

- Open source, designed for the web
- Good place to explore fonts
- Can be incorporated via the `{showtext}` package!

{showtext} example

```
devtools::install_github("yixuan/showtext")

library(showtext)

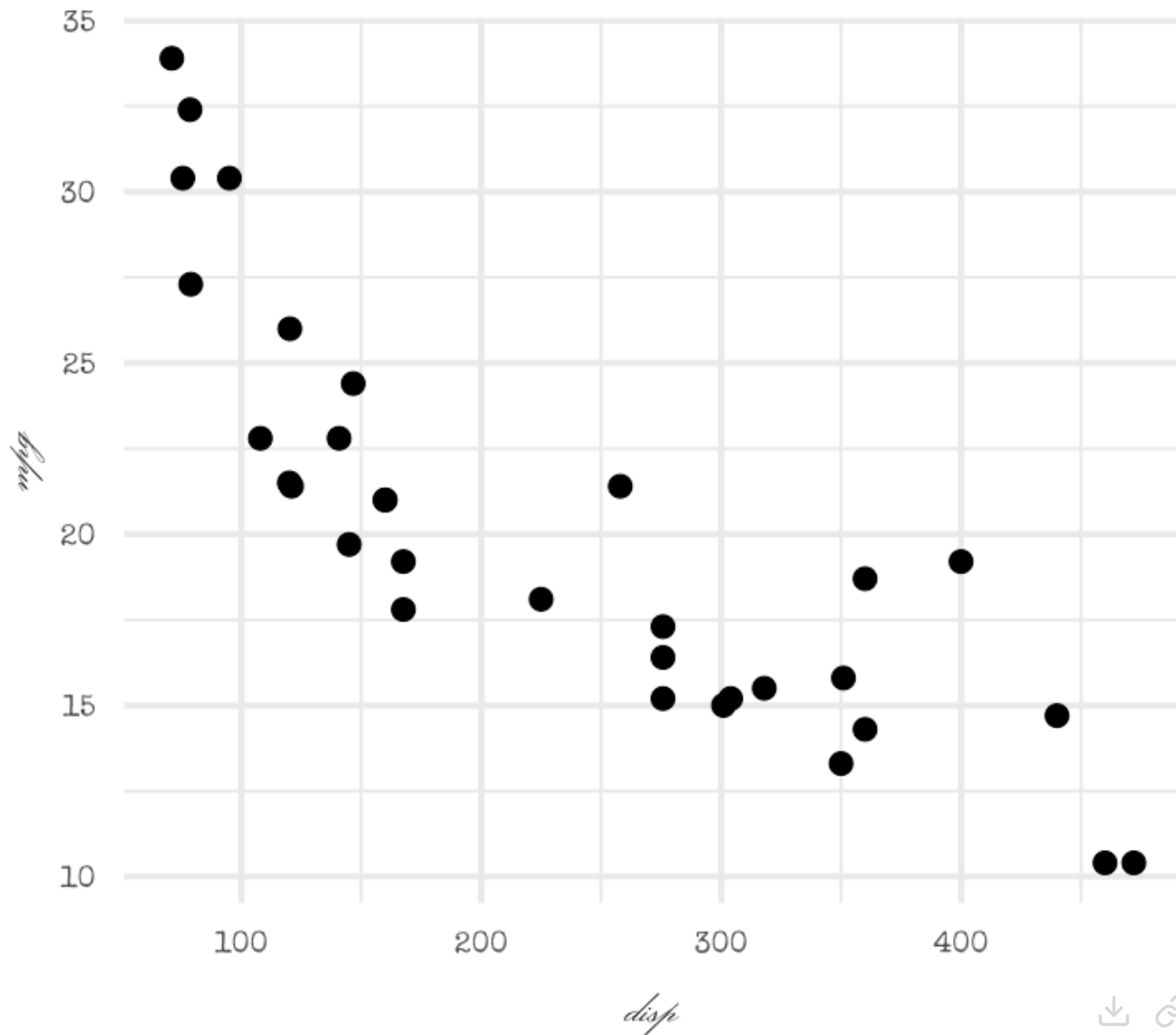
font_add_google('Monsieur La Doulaise', "mld")
font_add_google('Special Elite', "se")

showtext_auto()

ggplot(mtcars, aes(displacement, mpg)) +
  geom_point() +
  labs(title = "An amazing title",
        subtitle = "with the world's most boring dataset") +
  theme(plot.subtitle = element_text(size = 18, family = "se"),
        plot.title = element_text(size = 22, family = "mld"),
        axis.title = element_text(size = 18, family = "mld"),
        axis.text.x = element_text(size = 12, family = "se"),
        axis.text.y = element_text(size = 12, family = "se"))
```

An amazing title

with the world's most boring dataset



Try experimenting with new fonts!

- Create a simple plot
- Change the font to something on your computer (e.g., "Arial")
- Try importing and using a google font with **showtext**
- Try using different fonts for the title and subtitle

Why fonts matter

A few examples of epic fails

Megaflicks - LOL



The image features two light gray sticky notes with yellow tabs at the top, set against a dark gray background. The left note contains the text 'I will always find you' in a red, lowercase, cursive script font. The right note contains the text 'I WILL ALWAYS FIND YOU' in a red, uppercase, all-caps sans-serif font. Below the notes, the text 'FONT MATTERS.' is written in a white, uppercase, sans-serif font.

I will always
find you

I WILL ALWAYS
FIND YOU

FONT MATTERS.

You'll Always
Be Mine

YOU'LL
BE ALWAYS
MINE

Quick aside

Change the font of your R Markdown!

Create a CSS code chunk - write tiny bit of CSS - voila!

```
@import url('https://fonts.googleapis.com/css?family=Akronim&dis  
body {  
  font-family: 'Akronim', cursive;  
}
```

See the CSS slides for more information.

Render!

Untitled

Daniel Anderson

2/18/2020

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more information, see <http://rmarkdown.rstudio.com>.

When you click the Knit button a document will be generated that includes both content as well as the output of any embedded R code chunks like this:

```
summary(cars)
```

```
##           speed           dist
##  Min.      : 4.0      Min.      : 2.00
##  1st Qu.:12.0      1st Qu.: 26.00
##  Median :15.0      Median : 36.00
##  Mean   :15.4      Mean   : 42.98
##  3rd Qu.:19.0      3rd Qu.: 56.00
##  Max.    :25.0      Max.    :120.00
```

Including Plots

Aside

I actually did this for the table slides to make them a bit smaller!

```
24
25 ▾ ````{css echo = FALSE}
26 ▾ table {
27     font-size: 1rem;
28 ▲ }
29 ▲ ````
30
```

Resource for learning more

- I'm not an expert on fonts. I have mostly just picked what looks nice to me.
- Consider the accessibility of the font ([good resource here](#))* But don't get sucked into the rabbit hole of fonts!



Best I've heard of is practical typography



BUTTERICK'S PRACTICAL TYPOGRAPHY

2ND EDITION

Typography in ten minutes

Summary of key rules

Start

foreword by Erik Spiekermann

introduction

how to use this book

acknowledgments

about Matthew Butterick

legal

Please pay for this book

how to pay for this book

why you should pay

MB fonts

Why typography matters

what is typography?

who is typography for?

why does typography
matter?

what is good typography?

where do the rules come
from?

Type composition

straight and curly quotes

ellipses

Identify fonts

Use others work to help you - I found the font for these slides from Daniel's theme and he used one that he liked.

Use google chrome's developer tools to help!

Also consider downloading fonts (from google or wherever) and using them directly.

Check out this [great blog post](#) by June Choe.

Dashboards!

Two approaches to dashboards in R

	flexdashboard (R Markdown)	Quarto Dashboards
Format	<code>.Rmd</code>	<code>.qmd</code>
Available since	2016	Quarto v1.4 (2024)
Layout syntax	Markdown headers + <code>{data-width}</code>	Markdown headers + YAML attributes
Interactivity	<code>runtime: shiny</code>	<code>server: shiny</code>
Theming	Bootstrap themes, custom CSS	<code>_brand.yml</code> , Bootstrap, CSS
Value boxes	<code>flexdashboard::valueBox()</code>	Native chunk option <code>## content: valuebox</code>
Status	Stable, maintained	Actively developed

We'll cover **both** — flexdashboard first (since we're in R    25 / 78)

Part 1:

flexdashboard

The definitive source!

<https://rmarkdown.rstudio.com/flexdashboard/>

flexdashboard for R

Home

Using

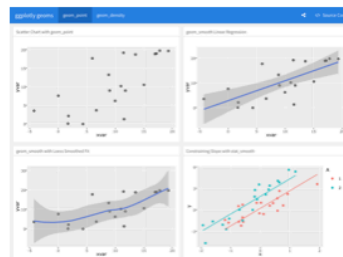
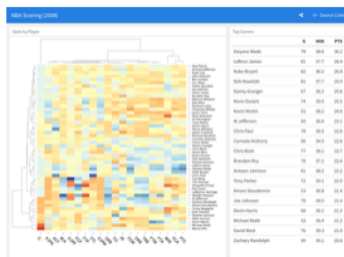
Shiny

Layouts

Examples

flexdashboard: Easy interactive dashboards for R

- Use [R Markdown](#) to publish a group of related data visualizations as a dashboard.
- Support for a wide variety of components including [htmlwidgets](#); base, lattice, and grid graphics; tabular data; gauges and value boxes; and text annotations.
- Flexible and easy to specify row and column-based [layouts](#). Components are intelligently re-sized to fill the browser and adapted for display on mobile devices.
- [Storyboard](#) layouts for presenting sequences of visualizations and related commentary.
- Optionally use [Shiny](#) to drive visualizations dynamically.



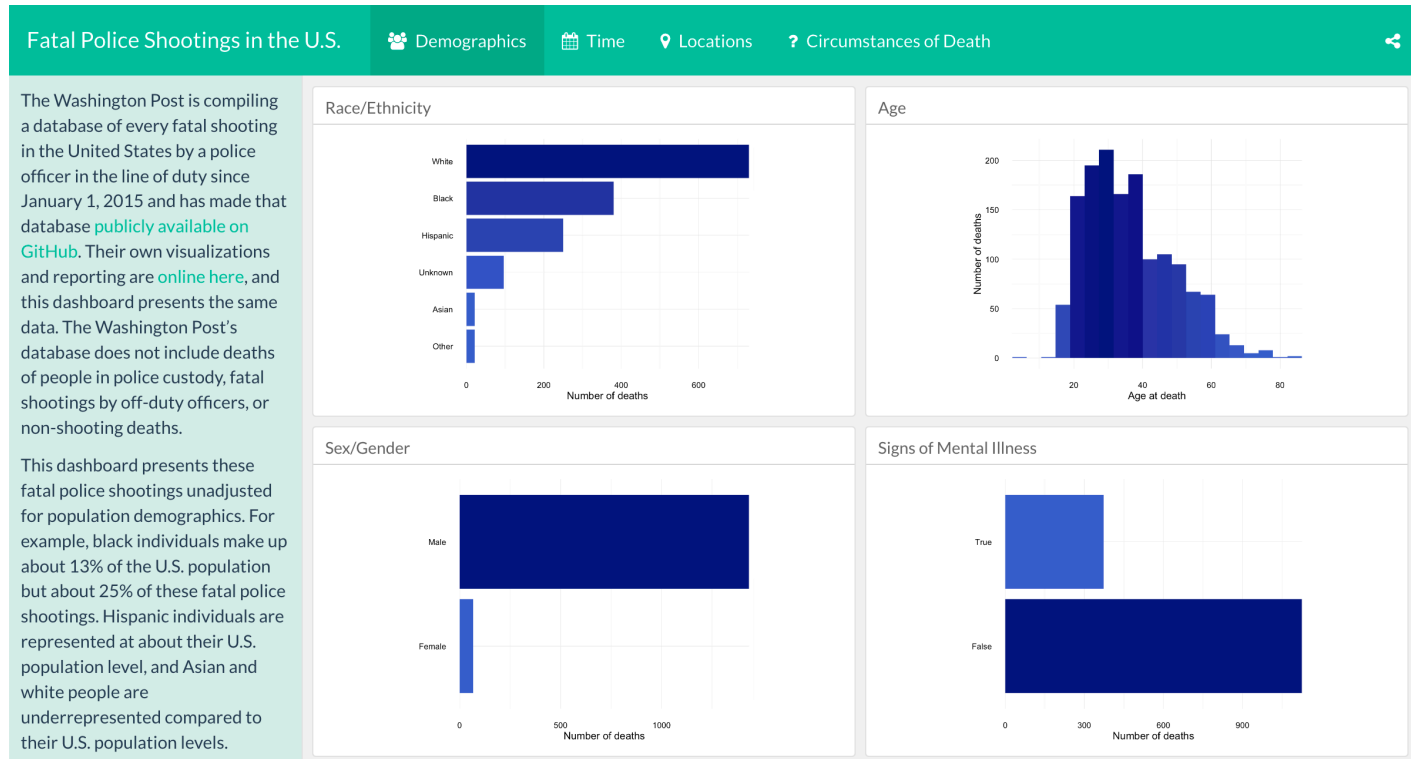
Getting Started

Install the **flexdashboard** package from CRAN as follows:

```
install.packages("flexdashboard")
```

{width="75%"}

Example



By [Julia Silge](#) (see the [blog post](#), [dashboard](#), and [source code](#))

Credit: This example from [Alison Hill's rstudio::conf\(2019L\) class](#)



Another example



{width="75%"}

One more...

By Jennifer Thompson (see the [blog post](#), [dashboard](#), and [source code](#))

Step 1: Create a new flexdashboard

```
#install.packages("flexdashboard")
```

New R Markdown

-  Document
-  Presentation
-  Shiny
-  From Template

Template:

 [Using R Markdown Templates](#)

Custom theming	{bslib}
Legacy custom theming	{bslib}
Real-time theming	{bslib}
Flex Dashboard	{flexdashboard}
Business Card	{pagedown}
HTML Letter	{pagedown}
HTML Poster (Jacobs University)	{pagedown}
HTML Poster (the ReLaXed style)	{pagedown}
HTML Resume	{pagedown}

Step 2: Look at the default YAML

When you create a new flexdashboard from the template, you get this:

```
---  
title: "My Dashboard"  
output:  
  flexdashboard::flex_dashboard:  
    orientation: columns  
    vertical_layout: fill  
---
```

Two key YAML options to know right away:

- **orientation**: **columns** (default) or **rows** — controls how content is laid out
- **vertical_layout**: **fill** (stretch to fit) or **scroll** (allow scrolling)

Step 3: Knit right away!

Before adding anything, **knit the template** so you know what the default looks like.

You should see a page with placeholder content split into two columns.

Now let's understand *why* it looks the way it does...

Understanding the layout model

flexdashboard uses **markdown headers** to define structure:

Header Level	What it creates
# Page Title	A new page (tab in the nav bar)
Column or Row (with dashes)	A new column or row
### Box Title	A new card/box within the column/row

Think of it as nesting: **Pages** > **Columns/Rows** > **Boxes**

Columns

- Define new column with

Column

- Optionally specify the width with `{data-width}`
- Annoyingly, be careful with spacing!

`{data-width=650}` will work

but

`{data-width = 650}` will not work

Step 4: A two-column layout

Here's the pattern you'll use most often:

```
Column {data-width=650}  
-----  
### Chart A  
< r code chunk >  
  
Column {data-width=350}  
-----  
### Chart B  
< r code chunk >  
  
### Chart C  
< r code chunk >
```

- The left column gets 65% of the width, with one chart
- The right column gets 35%, split evenly between two charts

New boxes

```
### Box title
```

```
< r code chunk >
```

- Each time you add a box it will split the area evenly among all the boxes in that column

Thinking in rows

- Change the YAML to

```
output:  
  flexdashboard::flex_dashboard:  
    orientation: rows
```

- Now `###` will create a new column *within* a row
- Add new rows with

Row

- Modify height with `{data-height=XXX}`

Step 5: Add a second page

You can easily specify multiple pages by just specifying a Level 1 Header

```
# Page 1
```

```
Column {data-width=650}
```

```
-----
```

```
### Chart A
```

```
< r code>
```

```
Column {data-width=350}
```

```
-----
```

```
### Chart B
```

```
< r code>
```

```
### Chart C
```

```
< r code>
```

```
# Page 2
```

Step 6: Make it interactive with tabsets

This is actually a standard R Markdown feature, but you can use it with flexdashboards as well

Column {.tabset}

Chart 1

< r code>

Chart 2

< r code>

Data Table

< r code>

No comma between multiple column arguments

Good

Column {.tabset data-width=650}

bad

Column {.tabset, data-width=650}

Step 7: Add a sidebar

Julia Silge's dashboard had a nice sidebar where she explained things... You can have this too!

Sidebar Title {.sidebar}

Your text here. You can use markdown syntax, including [links](http://blah.com), *italics*, **bolding**, etc.

- Multiple pages? Change the separator to keep the sidebar global:

Sidebar {.sidebar}

=====

Your text here. You can use markdown syntax, including [links](http://blah.com), *italics*, **bolding**, etc.

A brief aside on interactivity

- Things like `reactable::reactable` and `plotly::ggplotly` can help give your dashboard some nice interactivity.

Steps to interactivity

With multipage layouts

Add `runtime: shiny` to your YAML

```
---
title: "My amazing dashboard"
runtime: shiny
output:
  flexdashboard::flex_dashboard:
    orientation: columns
    vertical_layout: fill
---
```

Save your interactive piece into an object, and call the corresponding `render*` function.

```
p <- ggplot(...)
renderPlotly(p)
```

```
tbl <- reactable(...)
renderReactable(tbl)
```

Icons

- Probably not the most important thing, but fun
- Use Font awesome!

```
# Years {data-icon="fa-calendar"}
```

HTML Widgets

Add a touch of interactivity

- Plenty of HTML widgets for R out there (see https://www.htmlwidgets.org/showcase_leaflet.html)
- {plotly} is cool

```
#install.packages("plotly")  
library(plotly)  
p <- ggplot(mpg, aes(displ, cty)) +  
  geom_point() +  
  geom_smooth()  
  
ggplotly(p)
```

Including Text

- If you want to include text about an overall figure, just put the text in the R Markdown doc like you normally would

```
# Base {data-icon="fa-calendar"}
```

```
Here's a description about the plot that follows
```

```
### A base R plot
```

```
< r code>
```

What if you have tabsets?

- Works great if you want to describe all the plots/tables/content in the tabset
- If you want to provide text for an individual plot, use >

```
Column {.tabset data-width=350}
```

```
-----
```

```
This text will describe the full tabset
```

```
### Chart 1
```

```
< r code>
```

```
> Here's some text for Chart 1
```

```
### Table 1
```

```
< r code>
```

```
> Here's some text for Table 1
```

Storyboarding

- A little bit advanced, but pretty cool
- First, change the YAML

```
output:  
  flexdashboard::flex_dashboard:  
    storyboard: true
```

```
# Method {.storyboard}
```

```
### Sample Descriptives {data-commentary-width=400}
```

```
< r code>
```

```
***
```

This is some text describing what's going on with the sample, and how we moved from the raw data to the analytic sample.

```
### Correlation Matrix {data-commentary-width=200}
```

```
< r code>
```

```
***
```

There is less to say here so I made the commentary box smaller

```
# Results {.storyboard}
```

```
### Plot 1 {data-commentary-width=600}
```

```
< r code>
```

```
***
```

Lots to say here. There is important

```
### Plot 2 {data-commentary-width=200}
```

```
***
```



Customization

- Add font-awesome stuff
- Change the theme

```
flexdashboard::flex_dashboard:  
  theme: readable
```

CSS

More on this in later slides

Change the navigation bar to bright pink with thin blue border

```
.navbar-inverse {  
  background-color: #FE08A5;  
  border-color: #0822FE;  
}
```

Save the previous code in "custom.css" then specify in the YAML

```
flexdashboard::flex_dashboard:  
  css: custom.css
```

Making sure "custom.css" is in the same directory as your flexdashboard Rmd.

Add a logo and favicon

```
output:  
  flexdashboard::flex_dashboard:  
    logo: logo.png  
    favicon: favicon.png
```

Deploying flexdashboard to GitHub Pages

Since flexdashboard knits to a **single HTML file** (self-contained by default), deployment is simple:

Option 1: Manual (quick and dirty)

1. Knit your `.Rmd` → produces a `.html` file
2. Rename it to `index.html`
3. Push to a GitHub repo
4. Go to *Settings* > *Pages* → set source to the branch/folder containing `index.html`

Deploying flexdashboard to GitHub Pages

Option 2: `docs/` folder (cleaner)

1. Add `knit_dir` or just save/move your `.html` into a `docs/` folder
2. Push to GitHub
3. In *Settings* > *Pages*, set source to `/docs` on your main branch

Deploying flexdashboard to GitHub Pages

Option 3: GitHub Actions (automated)

- Use a workflow that runs `rmarkdown::render()` and deploys the output
- More setup, but re-renders automatically on every push
No special server needed — it's just an HTML file!

Data viz in the wild

Everett

Dodjivi

America and Rachel on Deck next week

Part 2: Quarto Dashboards

The modern approach (Quarto ≥ 1.4)

Why Quarto Dashboards?

flexdashboard is great, and it's not going anywhere. But Quarto Dashboards offer some compelling advantages:

- **Multi-language:** Works with R, Python, Julia, and Observable
- **Native value boxes:** No package function needed — just a chunk option
- **`_brand.yml` theming** (new in Quarto 1.6+): Define colors, fonts, and logos in one YAML file and apply across all your Quarto outputs
- **Light/dark mode toggle** (Quarto 1.7+): Built-in, automatic

The simplest Quarto dashboard

Create a `.qmd` file with this YAML:

```
---  
title: "My Dashboard"  
format: dashboard  
---
```

That's it. Every code chunk becomes a **card**.

Compare to flexdashboard, where you need:

```
output:  
  flexdashboard::flex_dashboard:  
    orientation: columns  
    vertical_layout: fill
```

Layout: Same mental model, cleaner syntax

The mapping from flexdashboard → Quarto:

flexdashboard	Quarto Dashboard
# Page Title	# Page Title
Column {data-width=650}	## Column {width=650}
### Chart Title	Code chunk with <code>## title: "Chart Title"</code>
{data-height=400}	{height=400}

Key difference: Quarto uses `##` for rows/columns (not the dashed line convention)

A Quarto dashboard layout

```
---  
title: "My Dashboard"  
format: dashboard  
---  
  
## Row {height=70%}  
  
```${r}  
#| title: "Main Chart"
ggplot(mtcars, aes(wt, mpg)) + geom_point()
```${r}  
  
## Row {height=30%}  
  
```${r}  
#| title: "Table"
knitr::kable(head(mtcars))
```${r}  
  
```${r}  
#| title: "Summary"
summary(mtcars$mpg)
```

# Value Boxes

---

One of the best features of Quarto Dashboards — **native value boxes**

In flexdashboard, you'd write:

```
flexdashboard::valueBox(42, caption = "Articles", icon = "fa-file")
```

In Quarto, you use chunk options:

```
```{r}
#| content: valuebox
#| title: "Total Articles"
#| icon: file-text
#| color: primary
list(value = 42)
```
```

Or even **pure markdown** (no code at all!):

# Value box colors

---

Quarto value boxes support semantic colors that adapt to your theme:

| Color                                  | Meaning                   |
|----------------------------------------|---------------------------|
| <code>primary</code>                   | Main brand color          |
| <code>secondary</code>                 | Subdued                   |
| <code>success</code>                   | Green — something good    |
| <code>info</code>                      | Blue — informational      |
| <code>warning</code>                   | Yellow/orange — caution   |
| <code>danger</code>                    | Red — alert               |
| <code>light</code> / <code>dark</code> | Light or dark backgrounds |

You can also use any hex color: `color: "#6C4675"`

# Value boxes: a full example row

---

```
Row {height="25%"}

```{r}
#| label: calculate-values
n_schools <- nrow(school_data)
avg_score <- round(mean(school_data$score), 1)
pct_proficient <- paste0(round(mean(school_data$proficient) * 100)
```

```{r}
#| content: valuebox
#| title: "Schools in Sample"
#| icon: building
#| color: primary
list(value = n_schools)
```

```{r}
#| content: valuebox
#| title: "Avg. Score"
#| icon: bar-chart
#| color: info
```

Pages, Sidebars, and Tabsets

Multiple pages

```
# Overview
## Row
...

# Details
## Row
...
```

Sidebar (page-level)

```
# Page 1

## {.sidebar}
Filter controls or explanatory text here.

## Column
...
```

Theming with `_brand.yml`

New in **Quarto 1.6** and enhanced in **1.7–1.8**, `_brand.yml` lets you define your visual identity in one place.

Theming with `_brand.yml`

Create a `_brand.yml` file in your project root:

```
color:
  palette:
    uoregon-green: "#154733"
    uoregon-yellow: "#FEE123"
  foreground: "#333333"
  background: "#FFFFFF"
  primary: "#154733"

typography:
  base:
    family: "Source Sans Pro"
  headings:
    family: "Source Serif Pro"
    weight: 600

logo:
  medium: logo.png
```

Light/Dark mode (Quarto 1.7+)

Provide a light and dark theme and Quarto adds a **toggle** automatically:

```
---  
title: "My Dashboard"  
format:  
  dashboard:  
    theme:  
      light: flatly  
      dark: darkly  
---
```

Or with `_brand.yml` (Quarto 1.8+):

```
# _brand.yml  
color:  
  foreground:  
    light: "#333333"  
    dark: "#555555"
```

Accessibility checks (Quarto 1.8)

Add the `axe` option to your dashboard to run automated accessibility checks:

```
---  
title: "My Dashboard"  
format:  
  dashboard:  
    axe: true  
---
```

When you render, Quarto will show warnings directly in the preview if there are accessibility violations (missing alt text, color contrast issues, etc.)

This is especially useful for **education policy dashboards** that need to be accessible to broad audiences.

Quick theming with Bootstrap

Just like flexdashboard, Quarto dashboards support Bootswatch themes:

```
format:
  dashboard:
    theme: cosmo
```

Available themes: [cosmo](#), [flatly](#), [journal](#), [lumen](#), [minty](#), [pulse](#), [sandstone](#), [simplex](#), [united](#), [yeti](#), and more.

You can layer a custom SCSS file on top:

```
format:
  dashboard:
    theme: [flatly, custom.scss]
```

Navigation bar options

```
---
title: "Education Dashboard"
author: "Maithreyi Gopalan"
format:
  dashboard:
    logo: images/logo.png
    nav-buttons:
      - icon: github
        href: https://github.com/username/repo
      - icon: linkedin
        href: https://linkedin.com/in/username
---
```

Social links in the nav bar — no extra work needed!

Deploying Quarto dashboards

Because Quarto dashboards render to **static HTML** by default:

- ****Same:** Just like flexdashboard - rename index.html and push
- **GitHub Pages:** `quarto publish gh-pages`
- **Netlify:** Drag and drop the `_site/` folder, or connect to your repo
- **Posit Connect / Connect Cloud:** `quarto publish connect`

We will discuss other deployments next week!

flexdashboard → Quarto: migration cheatsheet

flexdashboard	Quarto Dashboard
<code>output:</code> <code>flexdashboard::flex_dashboard:</code>	<code>format: dashboard</code>
<code>orientation: rows</code>	<code>orientation: rows</code> (same!)
<code>Column {data-width=650}</code>	<code>## Column {width=650}</code>
<code>### Chart Title</code>	<code>## title: "Chart Title"</code>
<code>{.tabset}</code>	<code>{.tabset}</code> (same!)
<code>runtime: shiny</code>	<code>server: shiny</code>
<code>flexdashboard::valueBox()</code>	<code>## content: valuebox</code>
<code>css: custom.css</code>	<code>theme: [default, custom.scss]</code>

Summary

flexdashboard

- Mature, stable, great for R-only projects
- Rich ecosystem of examples and documentation
- If you're already comfortable with R Markdown, it's easy

Quarto Dashboards

- The future — actively developed with new features every release
- Multi-language support (R, Python, Julia)
- Native value boxes, `_brand.yml` branding, light/dark mode, accessibility checks
- Better if you're already using Quarto for other things

My recommendation: Learn both layouts (they're very similar!), stick to the one you are comfortable with.

Resources

- [flexdashboard docs](#)
- [Quarto Dashboards docs](#)
- [Mine Çetinkaya-Rundel's Build-a-Dashboard workshop](#)
- [Quarto 1.8 release notes](#) — brand extensions, accessibility
- [Quarto 1.7 release notes](#) — light/dark mode
- [_brand.yml](#) documentation

Next time

Websites w/Distill and Quarto
Deployment to GitHub Pages
More CSS customization